In [1]:
```
import matplotlib.pyplot as plt
import pandas as pd
import zipfile
import io
import urllib.request
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMa
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

In [2]:
```
# 1. Load the Bank Marketing dataset
url = "https://archive.ics.uci.edu/static/public/222/bank+marketing.zip"
with urllib.request.urlopen(url) as response:
    bank_zip = zipfile.ZipFile(io.BytesIO(response.read()))

csv_file = None
for name in bank_zip.namelist():
    if name.endswith("bank-full.csv"):
        csv_file = bank_zip.open(name)
        break
    if name.endswith("bank.zip"):
        nested_zip = zipfile.ZipFile(io.BytesIO(bank_zip.read(name)))
        csv_name = next(item for item in nested_zip.namelist() if item.en
        csv_file = nested_zip.open(csv_name)
        break

df = pd.read_csv(csv_file, sep=";")
df.head()
```

Out[2]:

	age	job	marital	education	default	balance	housing	loan	contact
0	58	management	married	tertiary	no	2143	yes	no	unknown
1	44	technician	single	secondary	no	29	yes	no	unknown
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown
4	33	unknown	single	unknown	no	1	no	no	unknown

In [3]:
```
X = df.drop("y", axis=1)
y = df["y"]

categorical_features = X.select_dtypes(include=["object"]).columns.tolist
numerical_features = X.select_dtypes(exclude=["object"]).columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        ("categorical", OneHotEncoder(handle_unknown="ignore"), categoric
        ("numerical", "passthrough", numerical_features)
    ]
)

X = preprocessor.fit_transform(X)
y = y.map({"no": 0, "yes": 1})
```

```
if hasattr(X, "toarray"):
    X = X.toarray()
```

```
/var/folders/m8/tptc_62s19lc7hlwxbpzq5lh0000gn/T/ipykernel_66239/2976110300.py:4: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical_features = X.select_dtypes(include=["object"]).columns.tolist()
```

In [4]:
```
# 2. Split data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [5]:
```
# 3. Create and train the Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)
```

Out[5]:

▾ GaussianNB ⓘ ?

▸ Parameters

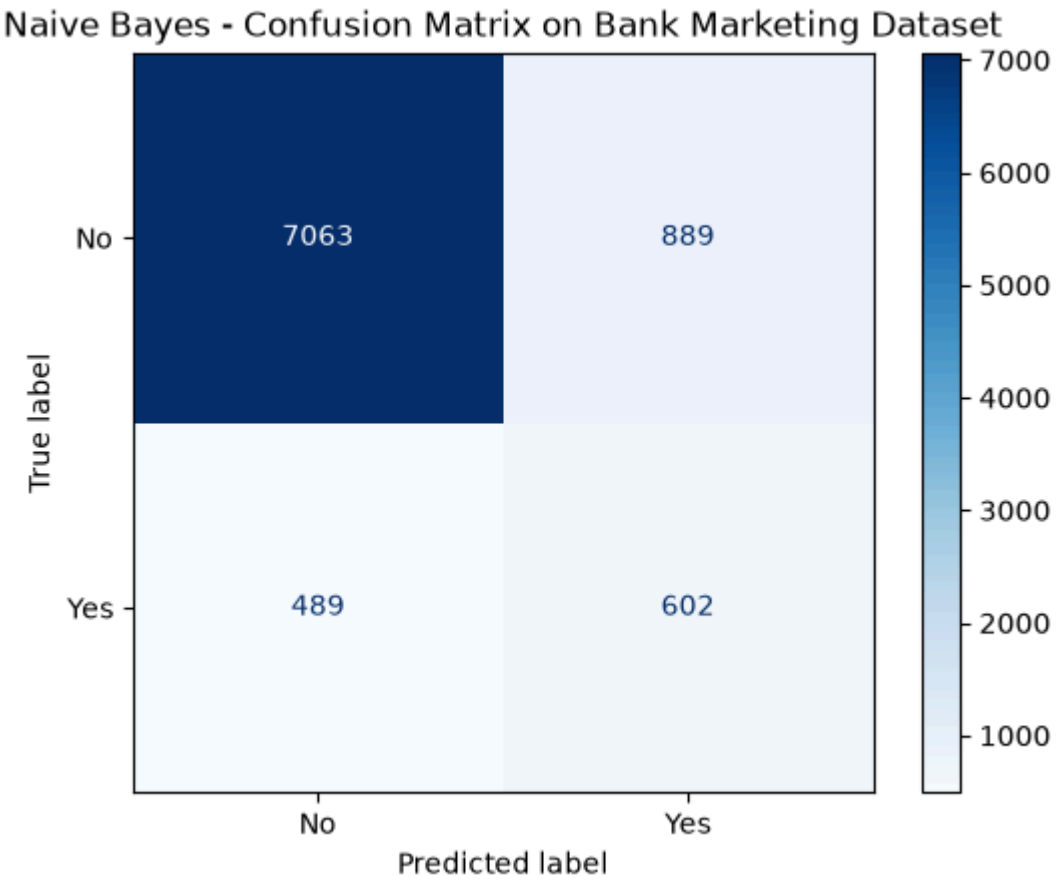
▸ Fitted attributes

In [6]:
```
# 4. Predict the term-deposit subscription for test data
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No",
disp.plot(cmap="Blues")
plt.title("Naive Bayes - Confusion Matrix on Bank Marketing Dataset")
plt.show()
```

```
Accuracy: 0.8476169412805485
```



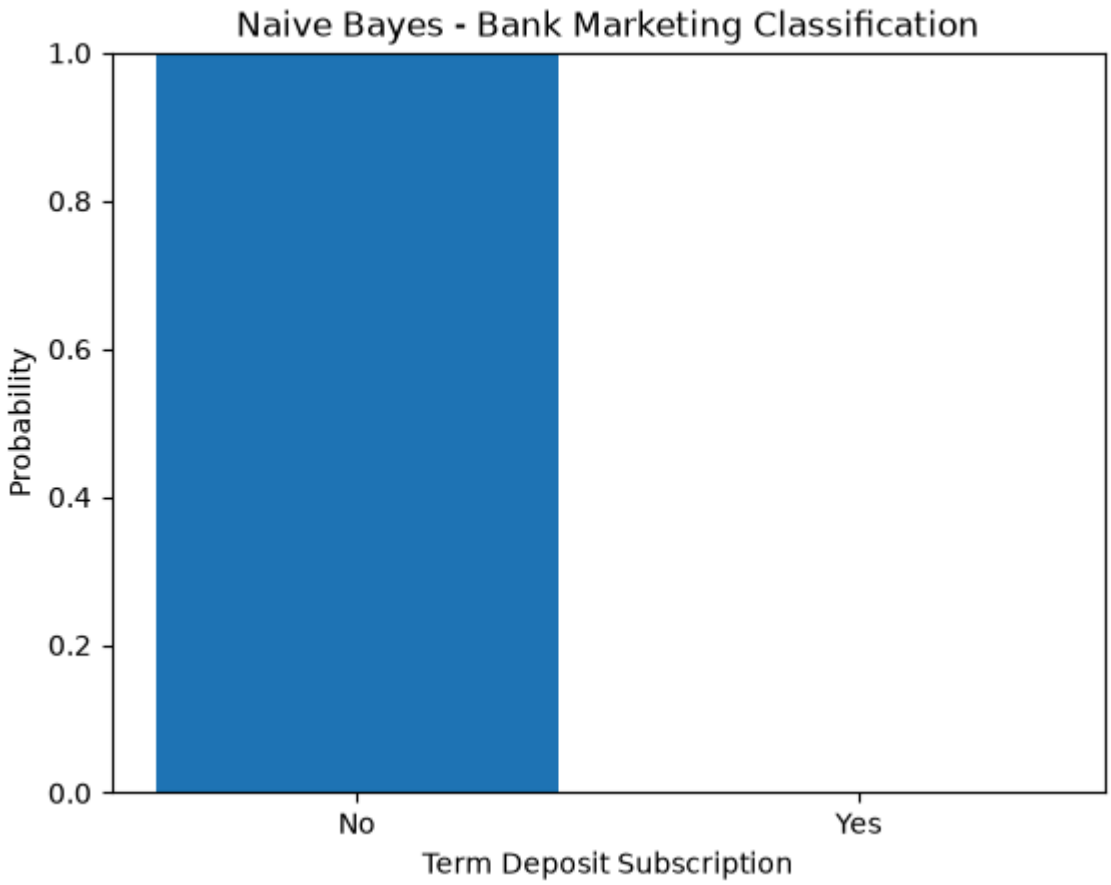
In [7]:
```
# 5. Select one test sample and predict its class
sample = [X_test[0]]
prediction = model.predict(sample)

# Calculate the probability of each target class
probability = model.predict_proba(sample)[0]

print("Predicted Term Deposit Subscription:", "Yes" if prediction[0] == 1
```

```
Predicted Term Deposit Subscription: No
```

In [8]:
```
# 6. Visualize the predicted probabilities of both classes
plt.bar(["No", "Yes"], probability)
plt.xlabel("Term Deposit Subscription")
plt.ylabel("Probability")
plt.title("Naive Bayes - Bank Marketing Classification")
plt.ylim(0, 1)
plt.show()
```


Naive Bayes - Bank Marketing Classification
1.0
0.8
0.6
0.4
0.2
0.0
Probability
No
Yes
Term Deposit Subscription

In [ ]: